DIGDIR/DIGDIR-AGENTS

Agents with their own computer

agentctl and the Sandbox SDK

Altinn Studio · DevOps

THE STARTING POINT

Coding agents on your laptop



Your credentials

The agent runs as you. Your tokens, your SSH keys, your cloud logins and your company network.



Permission prompts

Approve every command and you babysit the agent. With auto mode or no prompts, you trust it with your machine.



One at a time

Worktrees split the code, but there is still one set of ports, one Docker daemon and one localtest. A second agent means conflicts.

Either you do the verification, or you let the agent take more risks with your machine.

THE IDEA

Give every agent its own computer, with root, tools and network access, but none of your secrets.

The host keeps the secrets and decides **where each one may be sent**.

microVM per agent | mediated secrets | declarative manifests | any harness, any model

PART 1

Architecture

ARCHITECTURE

Two layers, one repo

Agent layer · `agentctl` + `agentd`
Agents, Sessions, harness adapters, control API, TUI, secret store

builds on ↓

Sandbox SDK · `sandbox`
Lifecycle, images, exec, files, terminals, network backends. No agent concepts.

sandbox-microsandbox
Backend + network enforcement

sandbox-authorization
Context-aware policy vocabulary

Microsandbox + libkrun
microVMs (KVM / Hypervisor.framework / WHP), Digdir forks

- Written in **Rust**, **local first** on Linux, macOS and Windows
- Every Sandbox is a **real VM** with its own kernel, not a container
- The SDK is **generic** and reused outside agentctl:

agentctl · local agents

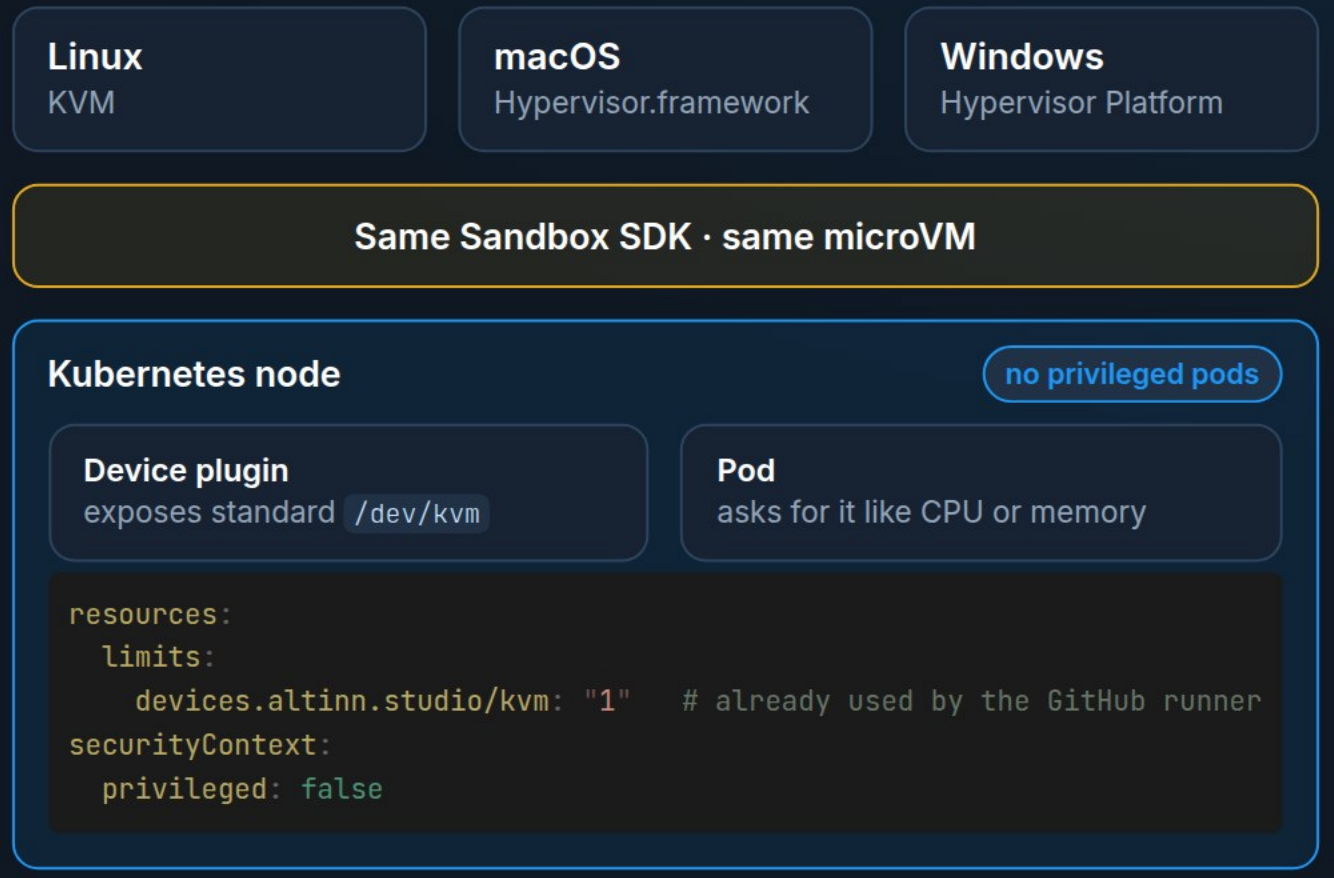
GitHub runner · altinn-studio CI

Studio assistant · opportunity

ARCHITECTURE · PRINCIPLE

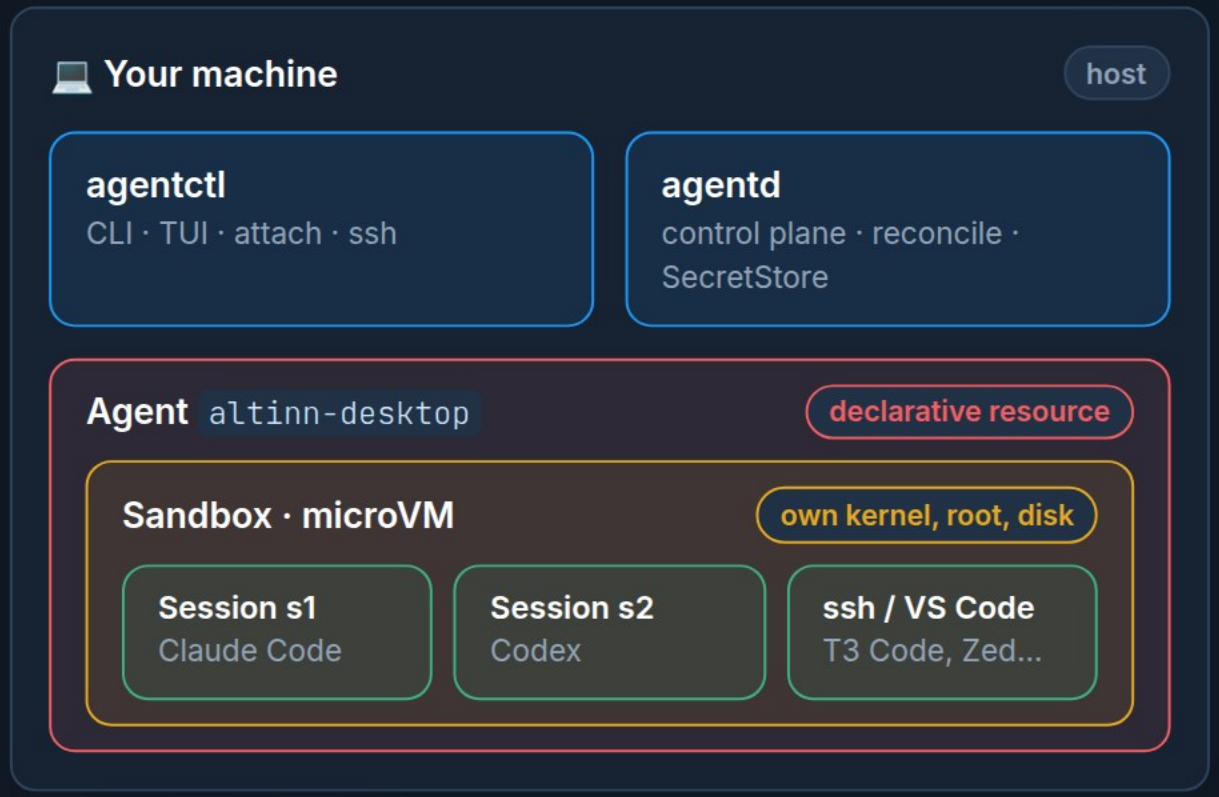
Local first as a forcing function

- Good architecture keeps **dependencies and coupling** to a minimum
- Three host OSes, three hypervisors, no cloud: the lower layers **have to** stay flexible
- Nothing in the SDK knows about agents, Kubernetes or a vendor
- Cloud is just **another host**: complexity that belongs to Kubernetes stays in Kubernetes



ARCHITECTURE

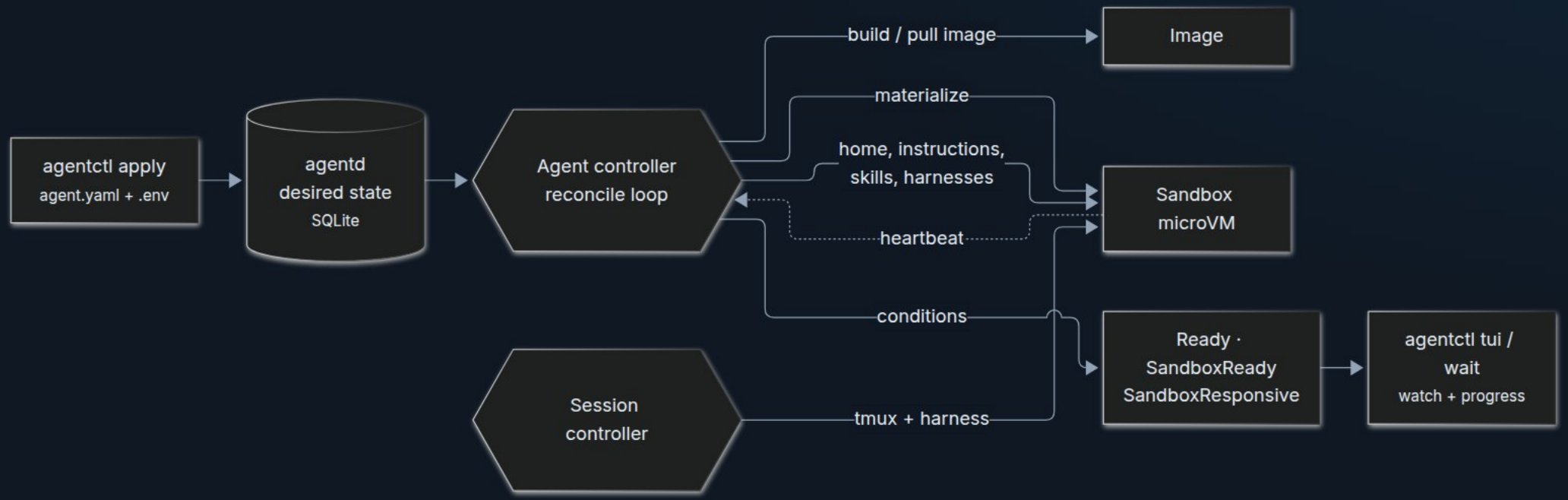
Host, Agent, Sandbox, Session



- **agentd** owns desired state and reconciles it, like a tiny Kubernetes controller
- An **Agent** owns one retained **Sandbox**: disk, volumes and identity survive restarts
- **Sessions** are durable harness processes in tmux. Detach, go idle, resume the conversation later

ARCHITECTURE

Declarative, reconciled



State, not streams

Progress is folded state. A late observer sees the same thing as an early one.

Survives restarts

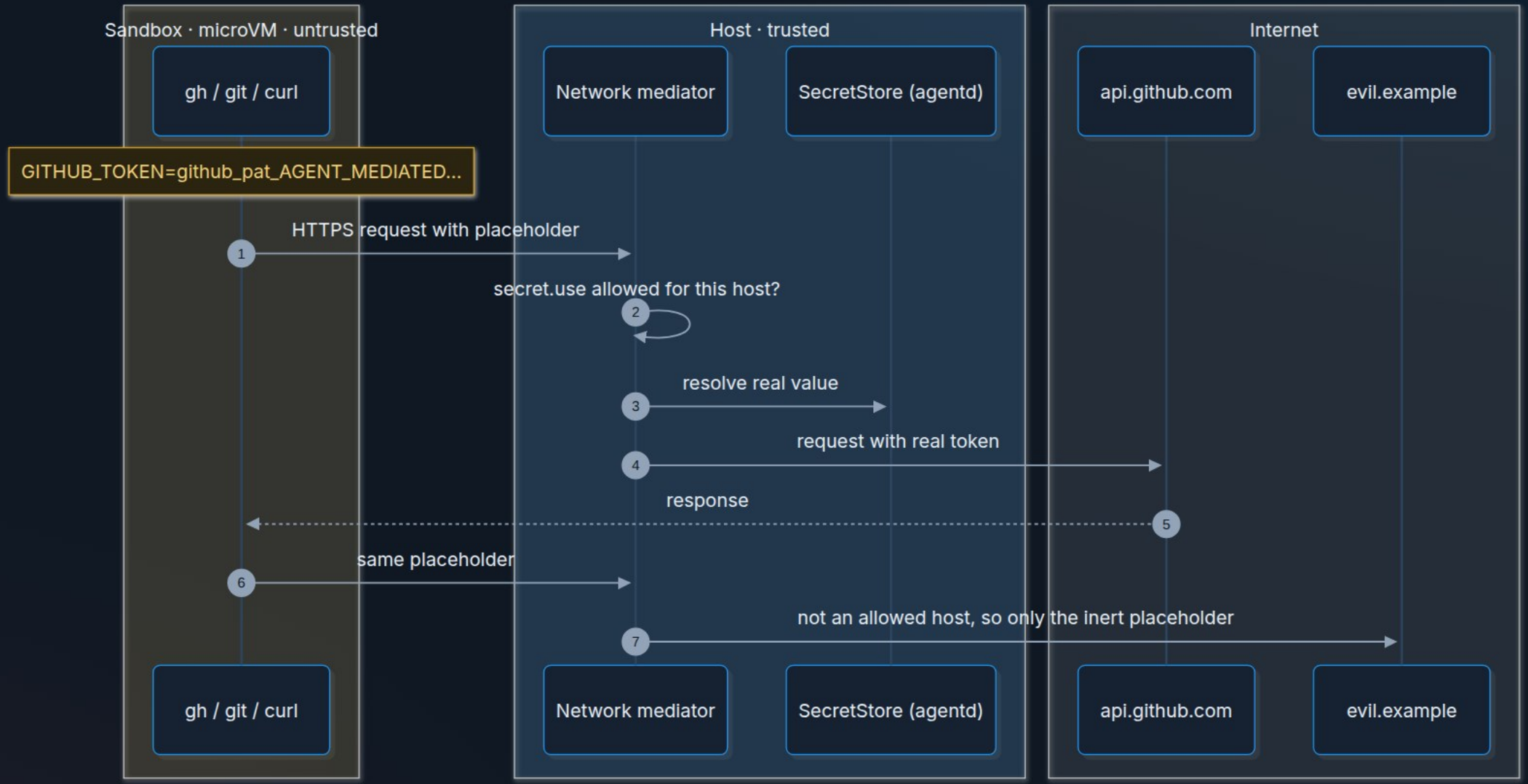
Desired state is persisted first. Periodic scans pick up lost wakeups.

Stalls are visible

Guest heartbeat stops for 15 s → `SandboxUnresponsive`, never a silent hang.

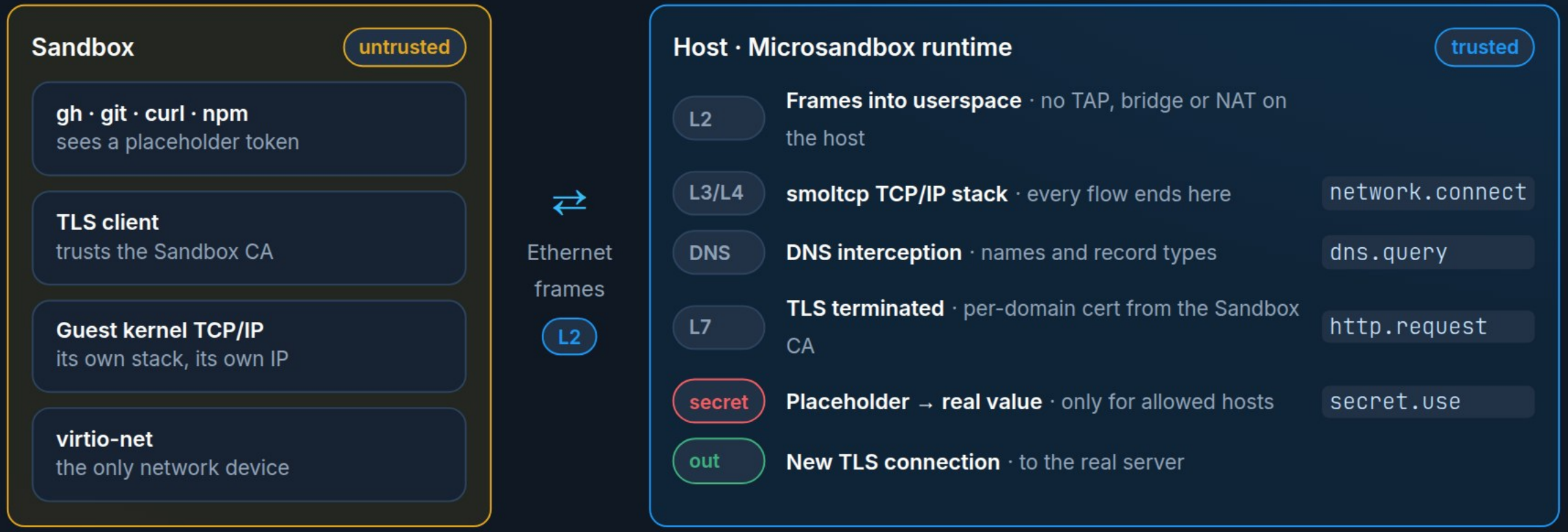
ARCHITECTURE · SECURITY

Secrets never enter the Sandbox



ARCHITECTURE · SECURITY

Every layer goes through the host



One policy vocabulary

`sandbox-authorization` for every check.

Context-aware

Hostname, method, path, secret location.

Fails closed

No policy or secret means nothing goes out.

ARCHITECTURE · OPEN QUESTION

Which identity does an agent act as?

Fine-grained PAT

Per repository and permission, expires. Tied to a person, so it stops when they leave.

Classic PAT

Broad scopes across every repo the person can reach. Hard to limit.

GitHub App

Short-lived installation tokens, owned by the org. Built for bots, but needs an admin to set up.

Own user or a bot account

Simple. A bot account is a seat that needs collaborator access, which also decides who can open PRs.

A real run

An agent signed in as a separate bot account, with read-only access to the repository. It forked the repo and tried to open the PR from the fork. GitHub refused: only collaborators may open PRs there. Access was granted afterwards and the PR went through.

Unsolved for us

Which option should Altinn use for agents? One identity per agent, one per team, or the person who started the task? Who grants and reviews the access?

ARCHITECTURE · SECURITY · HONEST LIMITS

What the Sandbox doesn't protect against

Protected

by design

Your machine
the agent has root only inside its own VM

Secret values
never in the VM, only sent to their allowed hosts

Other agents
separate VMs, disks and networks

The host network
only the Platform API is reachable

Not protected

needs other controls

Misuse of a token on its own hosts
anything the PAT allows on github.com, including a public gist

Data leaving without secrets
with `allow: all`, code can be posted to any host

Prompt injection
issues, web pages and dependencies can steer the agent

Bad code
review is still the gate, not the Sandbox

Sessions in one Sandbox
same user and tmux server, so one trust boundary

Narrow tokens, tighter `allow` lists and review shrink the gap. The policy engine is where stricter rules can go.

ARCHITECTURE

Harness-neutral

The platform doesn't care which agent runs inside.

Claude Code · mediated login, hooks, resume

Codex CLI · mediated ChatGPT login

Your harness · adapter: auth, config, launch, transcripts

Or bring your own UI over SSH.
`agentctl-<name>` is a plain OpenSSH alias.

T3 Code

Sandboxes as environments

VS Code / Zed

Remote-SSH into the VM

Claude desktop

Remote sessions

ChatGPT desktop

Remote sessions

PART 2

What is an agent definition?

AGENT DEFINITION

A folder in your repo

```
altinn-studio/agents/
├─ Dockerfile
├─ instructions.md
├─ skills/
├─ full/ · minimal/
└─ desktop/
   ├─ agent.yaml
   ├─ agent.worktree.yaml
   ├─ agent.nested.yaml
   ├─ agent.mine.yaml · gitignored
   ├─ desktop.md
   ├─ home/
   └─ .env.sample
```

Manifest · what to run, with which resources, secrets and network

Image · the toolbox: .NET, Node, Go, Podman, kind, studioctl, Chromium, a desktop…

Instructions · harness-neutral, installed as CLAUDE.md / AGENTS.md

Skills · reusable workflows: PR evidence, computer use, app development

home/ + .env · dotfiles overlay · local values and tokens (stay on host)

AGENT DEFINITION · AGENTS/DESKTOP/AGENT.YAML

The machine

```yaml
apiVersion: agents.platform/v1alpha1
kind: Agent
metadata:
  name: altinn-desktop
spec:
  sandbox:
    image:
      type: reference
      reference: ghcr.io/altinn/altinn-studio/agent-desktop:latest
    platform: { os: linux }
    resources:
      cpu: "4"
      memory: "8Gi"          # the browser needs it; the desktop idles at ~40 MiB
      rootFilesystem:
        capacity: "64Gi"
        mode: direct
    initSystem: image        # the image's systemd: desktop, VNC, git clone, studioctl login
    mounts:
      - type: tmpfs          # roomier /tmp for builds and Chromium's shared memory
        target: /tmp
        capacity: "2Gi"
```

AGENT DEFINITION · AGENTS/DESKTOP/AGENT.YAML

What the agent knows and runs

```yaml
home:
  source: home                # dotfiles overlay onto /home/agent, reapplied every pass
instructions:                 # → ~/.claude/CLAUDE.md and ~/.codex/AGENTS.md
  - source: ../instructions.md
  - source: ../full/environment.md
  - source: desktop.md        # "you have a desktop on :1, drive it with …"
skills:
  - source: ../../src/cli/resources/agent/skills/app-development
    name: altinn-studio-app-development
  - source: ../skills/pr-evidence
  - source: ../skills/computer-use
  - source: ../../.claude/skills/changelog
  - source: ../../.claude/skills/tekstforfatter-docs
  - source: ../../.claude/skills/text-content-review
harnesses:
  - type: claudeCode
    auth: mediated            # login stays on the host
    default: true
    defaults:
      model: fable
  - type: codex
    auth: mediated
    optional: true            # skipped if you have no Codex login
```

AGENT DEFINITION · AGENTS/DESKTOP/AGENT.YAML

Secrets, network and access

```yaml
environment:                # plaintext, copied from .env
  - name: GIT_USER_NAME
  - name: GIT_USER_EMAIL
secrets:
  - environment: GITHUB_TOKEN
    placeholder: github_pat_AGENT_MEDIATED_GITHUB_TOKEN
    allowedHosts: [github.com, api.github.com,
                   gist.github.com, uploads.github.com]
  - environment: AZURE_DEVOPS_PAT
    optional: true
    allowedHosts: [dev.azure.com]
  - environment: STUDIO_DEV_API_KEY
    optional: true
    allowedHosts: [dev.altinn.studio]
  # … staging + prod Studio keys
access:
  - type: ssh               # agentctl-altinn-desktop for any SSH client
  - type: vnc               # the desktop, in a browser or a VNC client
network:
  mode: mediated
  allow: all
  deny: [metadata.google.internal]
```

AGENT DEFINITION

Variants: one agent, many shapes

```yaml
# agent.worktree.yaml
apiVersion: agents.platform/v1alpha1
kind: AgentVariant
extends: agent.yaml
metadata:
  name: altinn-desktop-worktree
spec:
  sandbox:
    mounts:
      - type: bind
        source: ../..   # your checkout
        target: /home/agent/code/altinn-studio
        readOnly: false
```

```yaml
# agent.nested.yaml
apiVersion: agents.platform/v1alpha1
kind: AgentVariant
extends: agent.yaml
metadata:
  name: altinn-desktop-nested
spec:
  sandbox:
    resources:
      cpu: "2"
      memory: "3Gi"
```

default
fresh checkout

worktree
your checkout, mounted

nested
agent inside an agent

nested-build
image built from branch

AGENT DEFINITION · CONVENTION

Your own defaults: `agent.mine.yaml`

```yaml
# agents/desktop/agent.mine.yaml
apiVersion: agents.platform/v1alpha1
kind: AgentVariant
extends: agent.yaml          # or agent.worktree.yaml
metadata:
  name: altinn-desktop-mine
spec:
  sandbox:
    resources:
      cpu: "8"
      memory: "16Gi"
  harnesses:                 # arrays replace, not merge
    - type: claudeCode
      auth: mediated
      default: true
      defaults:
        model: opus
        effort: high
```

Gitignored by design

`agent.*.yaml` is ignored. Only the shared variants are committed.

Your machine, your taste

Bigger VM, another default model or effort, a different image tag. You don't need to fork the definition.

Shows up everywhere

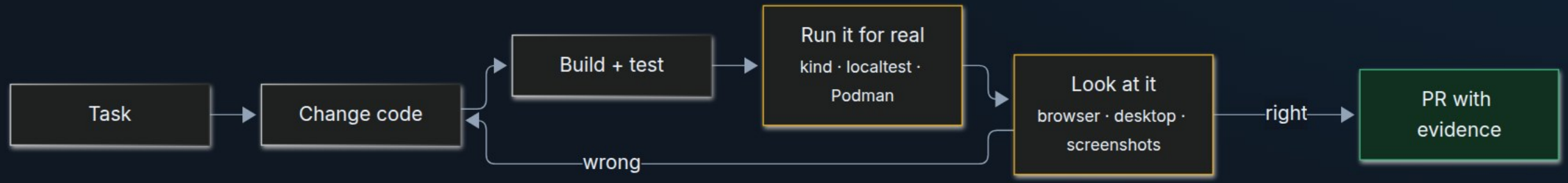
`agentctl apply --variant mine`, the TUI's create form and T3 Code's New Sandbox all find it.

PART 3

Productivity

PRODUCTIVITY

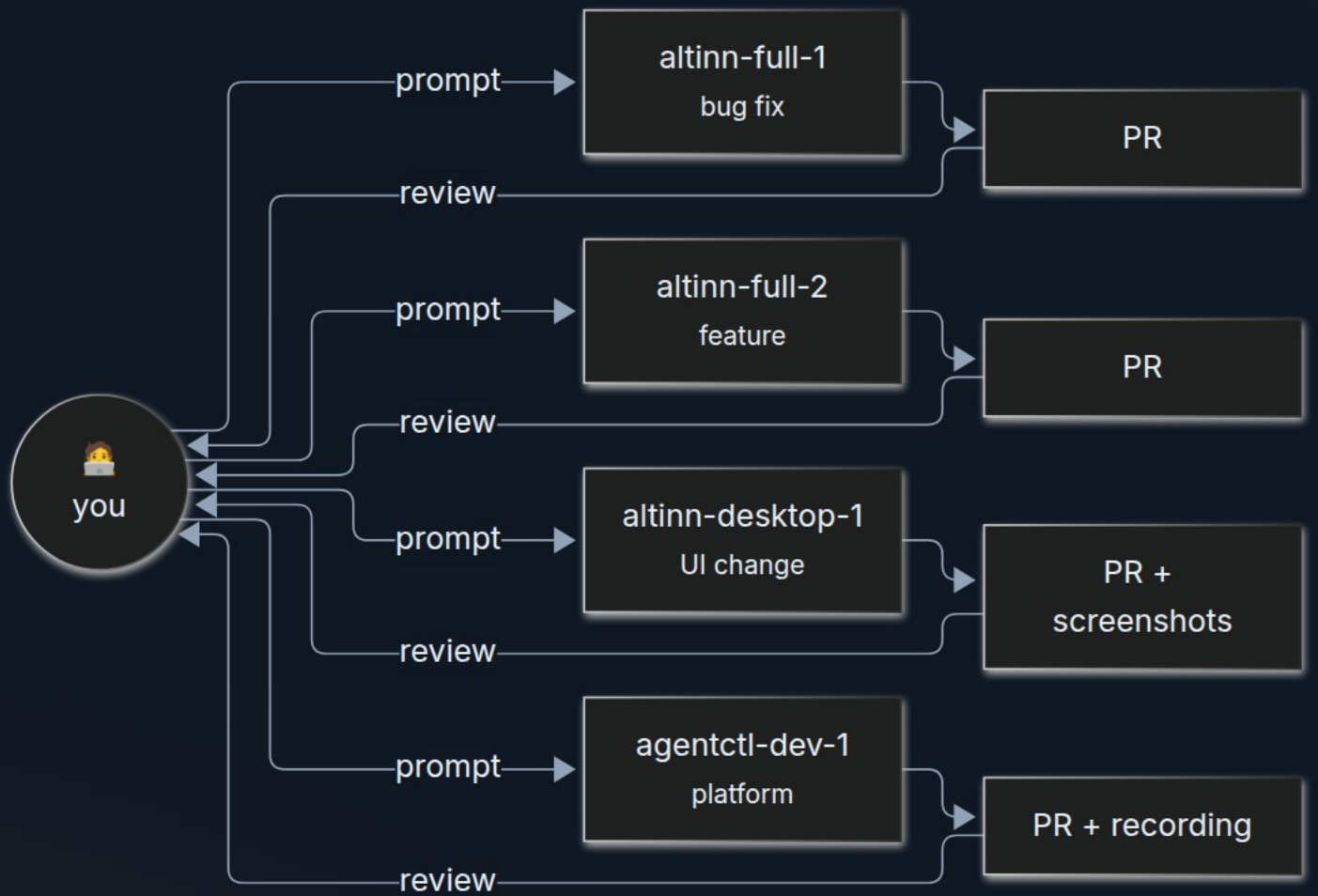
It owns its computer, so it can verify its own work



- **Root + no prompts:** `--dangerously-skip-permissions` is fine when the blast radius is a VM
- **Install what it needs:** `apt`, `podman`, a kind cluster, a database
- **Run the whole stack:** Studio, localtest, apps via `studioctl`
- **See the result:** Chromium, Playwright and a real desktop it can drive
- **Show you:**
  - screenshots and recordings attached to the PR
  - review in VS Code or another IDE over SSH
  - T3 Code's browser preview: open a page in the Sandbox, or watch the VNC desktop while it works

PRODUCTIVITY

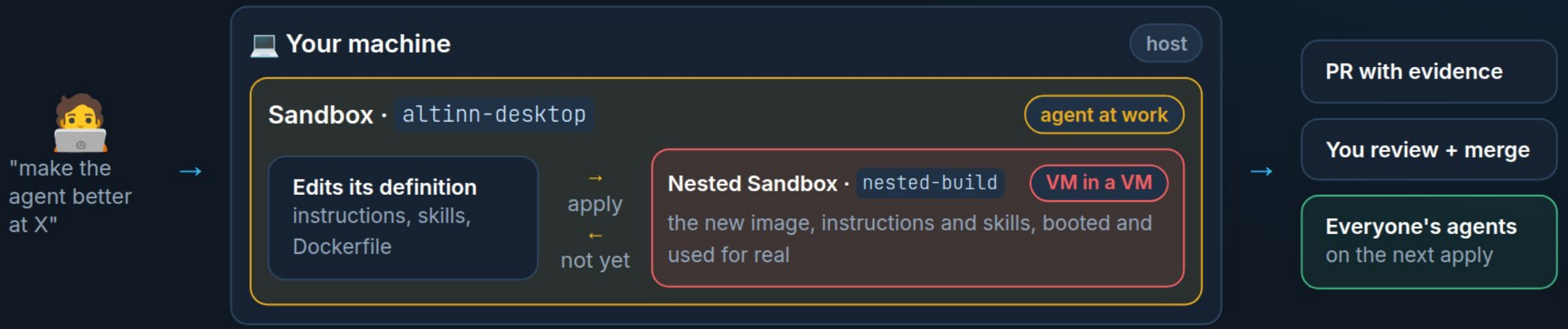
Scale out the human



- Every agent has **its own checkout, ports and cluster**, so they never conflict
- Agents **go idle and resume**, so long tasks keep their context
- You move from **typing code** to **framing tasks and reviewing**
- **Talk to them from anywhere**: TUI, SSH, VS Code, T3 Code

PRODUCTIVITY · NESTED AGENTS

Agents that improve the agent



Context and harness engineering

So far the main use of nested agents: the agent works on its own definition.

Tested for real

It boots the changed image, instructions and skills in a nested Sandbox and uses them.

Reviewed like code

You approve a PR with evidence, not a prompt someone tweaked by hand.

Nested virtualization

Tested on Linux hosts. macOS support is possible.

PRODUCTIVITY · A TRUE STORY

How this deck was made

scaffold

Empty deck

An agent in an Altinn Sandbox set up Slidev and served it to my laptop with `agentctl port-forward`.

debug

Forward refused

Slidev listened on IPv6 loopback only. The agent found it with `ss` and fixed the dev script.

research

Read the source

digdir-agents, altinn-studio and the Microsandbox network engine, to get the facts right.

verify

Looked at every slide

Headless Chromium screenshots after each change, to catch overflow and broken highlights.

deliver

PR #176

Blocked at first: the bot account had read-only access. Granted, then opened with screenshots.

My part: the topic, review comments and one permission change. No local setup.

PART 4

Same SDK, other workloads

ALTINN-STUDIO · SRC/CI/GITHUB-RUNNER

Sandboxed GitHub Actions runners



Ephemeral and isolated

One VM per job. Writable root, workspace and Docker data are private to the job.

Credentials stay out

The GitHub App key stays in the coordinator. The guest gets a one-time token and drops it before the runner starts.

Real Docker in CI

Containers inside a VM, not a privileged pod. Prepared images keep cold starts short.

OPPORTUNITY · NOT DECIDED

Studio assistant on the Sandbox SDK

Step 1 · Sandbox as a tool

middle step

Assistant (Python / LangGraph)
decides to upgrade the app

↓ HTTP tool call, a copy of the app in the request (multipart)

Coordinator-like service
spawns a Sandbox and unpacks the app in it

↓

`studioctl app upgrade v9`
runs as the tool, inside the Sandbox

↓

git patch
returned to the assistant and the user

Later · the agent in the Sandbox

long term

Studio Designer
a general assistant experience

↓

Pi / Pi-durable in the Sandbox
instead of Python / LangGraph

↓

Same building blocks
images, skills, mediated secrets, network policy

ALTINN STUDIO

What we get out of it

More done per developer

Parallel agents that verify their own work, so we review results instead of babysitting.

Safer by default

Agents never hold our tokens. Each secret is scoped to its hosts, and the blast radius is one VM.

Shared, reviewed setup

The Altinn agent lives in `altinn-studio/agents`. A better toolbox or skill helps everyone on the next apply.

One platform, many uses

Local agents and CI runners already share the Sandbox SDK and its hardening, and the assistant may be next.

TODO
PRs from agents / month

TODO
CI jobs on sandboxed runners

TODO
people using agentctl

ALTINN STUDIO · STRATEGY

Vendor-neutral by design

One vendor, all layers

coupled

- Their UI and cloud agents
- Their harness
- Their models
- Their sandboxes and runners
- Their identity and access model
- Your workflows, data and know-how, on their platform

Changing one layer means changing all of them.

Swappable layers, owned core

decoupled

UI	TUI · T3 Code · VS Code · Zed · SSH
Harness	Claude Code · Codex · + adapters
Model	whatever the harness supports
Sandbox SDK + agentctl	ours · open source
Definitions, skills, instructions	ours · in our repos
Authorization	our policy engine · workload identity on k8s
Runs on	laptops · our Kubernetes · any host with KVM

We have seen this before

Norwegian public sector and Microsoft: Office, Azure, Entra for identity and devices. Leaving takes years.

The stakes are going up

As more of our output runs through AI, owning the data, workflows and processes around it gets more valuable.

PART 5

From zero to a running prompt

ONBOARDING

Zero to a running prompt

1

Install

`install.sh`

Linux · macOS · Windows

2

Log in

`agentctl claude login`

(+ codex, optional)

3

Clone

`gh repo clone Altinn/altinn-studio`

4

Tokens

`.env.sample` → `.env`

GitHub PAT, Git identity

5

Create

`agentctl tui` → `c`

pick agent + variant

6

Prompt

`n` new Session

type your task ⏎

One-time

Steps 1–4: install, log in, clone and add tokens. A few minutes.

Per agent

Step 5: the first image pull takes a while. Later agents reuse the cached image.

Per task

Step 6: a new Session, or a new thread in T3 Code. Detach with `Ctrl-b d`.

ONBOARDING · SAME THING IN THE TERMINAL

Or without the TUI

```
# 1-2. Install agentctl + agentd, log in to Claude Code (login stays on the host)
curl -fsSL https://raw.githubusercontent.com/digdir/digdir-agents/main/agentctl/install.sh | sh
agentctl claude login

# 3. Get the agent definitions
gh repo clone Altinn/altinn-studio && cd altinn-studio/agents/desktop

# 4. Tokens: real values stay on the host, the VM only sees placeholders
cp .env.sample .env
$EDITOR .env

# 5. Create the Agent and wait until it is Ready
agentctl apply --wait
agentctl get agents

# 6. Start a Session and attach (detach with Ctrl-b d)
agentctl attach session/s1
```

DEMO

`agentctl tui` & T3 Code

TUI

Create an agent, start a Session, open it in VS Code or the desktop, port-forward

T3 Code fork

Sandboxes as environments: new thread, PR watching, previews

Parallel

Several agents on several tasks, reviewed as PRs

HONEST LIMITS

What it costs

Laptop resources

`altinn-desktop` asks for 4 CPUs, 8 GiB memory and up to 64 GiB disk. Check your host before running several at once.

Model usage

Parallel agents multiply token use. Subscriptions and API budgets become a real line item.

Review load

Agents produce PRs faster than people review them. The bottleneck moves to the human.

Upkeep

Images, harness version bumps with live compatibility tests and the Microsandbox fork pin all need maintenance.

STATUS

Where it is now

Works today

- `0.1.0-preview.11`, with `agentctl self update`
- Linux, macOS and Windows hosts
- Claude Code and Codex, mediated logins
- Durable Sessions: idle, resume, archive
- TUI, SSH, VS Code/Zed, VNC desktop, port forwards
- Sandboxed GitHub runners in altinn-studio CI

Next

- Session content and prompt/steer/interrupt APIs
- A Platform API in the Sandbox for delegation and host plugins
- Studio assistant on the SDK? (opportunity)
- Cloud agents? Software factories?

THANKS

Questions?

github.com/digdir/digdir-agents · `agentctl/README.md`

github.com/Altinn/altinn-studio · `agents/` · `src/ci/github-runner`